HPC Project Roadmap

Protein Secondary-Structure Prediction on CB513

Serial + OpenMP + POSIX Threads + MPI + Hybrid + CUDA (remote)

Date: February 08, 2026
Target audience: EE7218/EC7207 student team (3 CPU-only laptops)

Why this topic: Real ML benchmark (CB513) + a credible paper that compares OpenMP vs pthreads on neural-network training.
Your hardware reality: three Intel laptops (16 threads each) with no NVIDIA GPU; CUDA experiments run on a lab/cloud GPU.
What you will submit: separate executables for serial, OpenMP, pthreads, MPI, hybrid (MPI+OpenMP), plus a CUDA version and an analysis report.

Table of Contents

1. Scope and alignment with your course rubric

Your course requires separate deliverables for serial, shared-memory, distributed-memory, hybrid programming, plus an analysis report that includes a parallelization diagram, an accuracy check against serial, and timing comparisons across thread counts and other parameters.

1.1 Deliverables checklist (what you will produce)

- Serial baseline executable: single-thread training + evaluation (Q3 accuracy).
- Shared-memory executables: (A) OpenMP version and (B) POSIX threads (pthreads) version.
- Distributed-memory executable: MPI data-parallel training across 3 laptops.
- Hybrid executable: MPI across laptops + OpenMP within each laptop (recommended).
- CUDA executable: GPU version run on a remote NVIDIA GPU (lab/cloud/Colab).
- Analysis report (PDF): diagrams + accuracy table + timing/speedup plots.

1.2 Paper precedent you will cite

This roadmap is inspired by Zhong et al. (2007), who parallelize neural-network training for protein secondary-structure prediction and compare pthreads against OpenMP. DOI: **10.1007/s11227-007-0100-1**. Springer page: Springer landing page.

2. System constraints and deployment plan

2.1 Your compute environment (hard constraints)

Team setup: three similar CPU-only laptops (no CUDA-capable NVIDIA GPU). Target: run OpenMP, pthreads, and MPI locally across the three machines; run CUDA experiments remotely.

2.2 Recommended execution split

- Local (3 laptops): serial, OpenMP, pthreads, MPI, hybrid (MPI+OpenMP).
- Remote GPU (lab/cloud/Colab): CUDA build + timing runs.
- Single shared codebase, multiple build targets (Makefile or CMake).

2.3 Toolchain links (official references)

OpenMP: specs and ref guides. MPI/OpenMPI: mpirun manual. Pthreads: man7 pthreads(7) and LLNL tutorial. CUDA: CUDA docs.

3. Data: CB513 + labels + what you will actually use

3.1 Dataset choice

Use CB513, a classic benchmark for secondary-structure prediction (513 non-homologous proteins). Prefer sources with clear provenance and stable hosting.

Recommended sources (pick one and document it in your report):

- CB513 original page (JPred legacy): JPred CB513.
- CB513 with precomputed features/profiles (NumPy arrays): GitHub bundle.

3.2 Ground-truth labels and Q3 metric

Secondary structure labels are typically derived from experimentally determined 3D structures using DSSP (Dictionary of Secondary Structure of Proteins). DSSP assigns 8-state labels; the paper maps them to 3-state labels and evaluates Q3 accuracy (per-residue percent correct).

DSSP reference: PDB-REDO DSSP.

3.3 Feature encoding (keep it student-feasible)

Start with one-hot (orthogonal) encoding of amino acids over a sliding window. The paper uses a window size of 13 residues and discusses additional encodings (BLOSUM62, hydrophobicity, PSSM).

- MVP (recommended): one-hot encoding, window=13, input dimension 20×13=260.
- Enhancement: add BLOSUM62 encoding as a second option (same pipeline; different input).
- Avoid for MVP: generating PSSM via PSI-BLAST (heavy preprocessing). Use prepackaged profiles only if your chosen bundle includes them.

4. Preprocessing pipeline (dataset -> training examples)

4.1 Tasks you must implement

- Parse sequences and per-residue labels (H/E/C).
- Apply 8-state -> 3-state mapping if needed (as in the paper).
- Generate sliding windows of length 13; predict the center residue.
- Encode each window into a fixed-length numeric vector (one-hot or BLOSUM62).
- Split into train/validation/test with documented protocol (or use bundle's splits).
- Write the processed dataset to a fast binary format to keep timing runs clean.

4.2 Sanity checks (do these before any parallel code)

- Count total proteins and residues; print class distribution H/E/C.
- Verify windowing at sequence edges (padding strategy) is consistent.
- Run a tiny training run (e.g., 1 epoch) and confirm loss decreases.
- Confirm deterministic behavior with a fixed RNG seed.

4.3 CLI design (helps grading)

Make every executable accept consistent flags, so experiments are scriptable and comparable.

```
Example CLI flags (all executables)
--data processed_dataset.bin
--epochs 10
--batch 128
--lr 0.01
--seed 123
--threads 8 (OpenMP / pthreads)
--out results/run1.json (write metrics + timing)
```

5. Serial baseline (the truth you compare everything against)

5.1 Baseline model (practical substitute for DBNN)

Implement a small feedforward neural network (MLP) with two hidden layers and a 3-class softmax output. This preserves the paper’s "two hidden layers" spirit while staying student-feasible.

5.2 Serial training loop tasks

- Forward pass: compute logits for each sample in the batch.
- Loss: cross-entropy.
- Backward pass: compute gradients for weights and biases.
- Update: SGD (or momentum SGD).
- Evaluation: compute Q3 on validation/test.

5.3 What to log

- Q3 accuracy (train/val/test).
- Time per epoch and total time.
- Compilation flags and CPU info for reproducibility.

6. Shared-memory parallelism A: OpenMP

6.1 Parallel strategy

Parallelize over samples (or mini-batches). Use per-thread gradient buffers and reduce at the end of each batch to avoid locks in the hot loop.

6.2 Implementation tasks

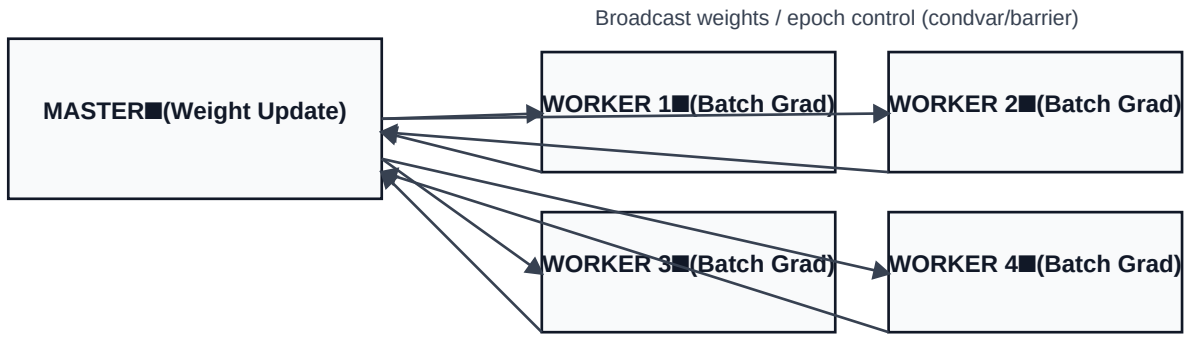
- Use a persistent OpenMP parallel region around the batch loop to reduce overhead.
- Allocate per-thread gradient buffers (weights and biases).
- Reduce (sum) gradients into a global gradient.
- Single-thread weight update (safe and simple).
- Verify Q3 matches serial within tolerance.

6.3 Key OpenMP settings to document

```
Build: g++ -O3 -march=native -fopenmp ...
Run: OMP_NUM_THREADS=1|2|4|8|16 ./train_omp ...
Pinning (optional): OMP_PROC_BIND=true OMP_PLACES=cores
```

7. Shared-memory parallelism B: POSIX threads (paper-style master/worker)

7.1 Architecture diagram (use this in your report)



Workers compute gradients on disjoint batches, master aggregates & updates

7.2 Task breakdown

- Master thread: owns the authoritative weights; performs the weight update step.
- Worker threads: compute partial gradients on disjoint chunks of the batch.
- Synchronization: barrier/condition-variable to start work and to signal completion.
- Reduction: master aggregates partial gradients then broadcasts the next iteration.

7.3 Implementation tasks

- Create threads once at startup (thread pool).
- Use pthread_mutex + pthread_cond (or pthread_barrier) to coordinate batches/epochs.
- Keep per-thread gradient buffers separate to avoid false sharing.
- Implement a clean shutdown (exit flag).
- Verify Q3 matches serial.

7.4 What to compare (OpenMP vs pthreads)

Show timing and speedup curves for both OpenMP and pthreads on the same workload and hyperparameters. Discuss overheads and synchronization costs.

8. Distributed-memory: MPI across 3 laptops

8.1 Cluster setup tasks

- Install OpenMPI on all machines; verify versions match.
- Set up passwordless SSH from the launch machine to the others.
- Create a hostfile listing the three hosts and available slots.
- Test with an MPI hello-world.

```
Example run (3 ranks across 3 laptops)
mpirun -np 3 --hostfile hosts.txt ./train_mpi --data processed.bin --epochs 10
--batch 256
```

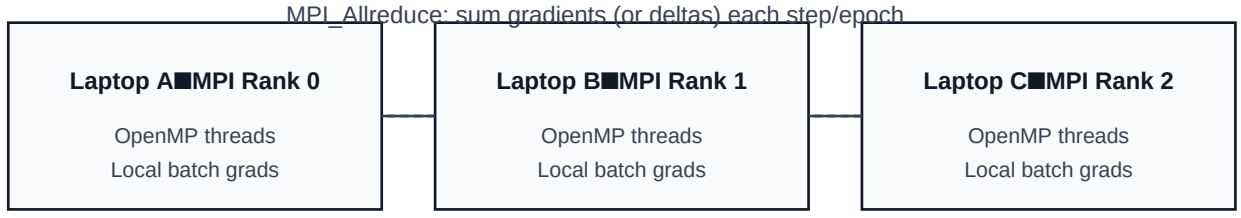
8.2 MPI training strategy (data parallel)

Use synchronous data-parallel SGD: each rank computes local gradients; combine with MPI_Allreduce; update weights identically on all ranks.

- Shard dataset by rank (static block partitioning is simplest).
- Allreduce gradients (sum) each step or each epoch.
- Barrier before timing; report max per-rank time per epoch.

9. Hybrid programming: MPI + OpenMP (recommended)

9.1 Architecture diagram (use this in your report)



Hybrid: MPI across laptops + OpenMP within each laptop

9.2 Practical choices to avoid oversubscription

Start with 1 MPI rank per laptop and 8 OpenMP threads per rank. Sweep threads; avoid ranks×threads > 16 per laptop.

```
Typical hybrid launch
export OMP_NUM_THREADS=8
mpirun -np 3 --hostfile hosts.txt ./train_hybrid_mpi_omp --data processed.bin
--epochs 10
```

10. CUDA plan (no local NVIDIA GPU)

10.1 What to implement on GPU

- GPU-accelerate dense layer forward/backward or batched GEMM.
- Option A: write CUDA kernels for GEMM + activation.
- Option B: call cuBLAS for GEMM and write small kernels for activation/softmax.

CUDA docs: CUDA Toolkit and cuBLAS.

10.2 Where you run it

- University GPU lab machine (best for stable timing).
- Cloud VM with NVIDIA GPU (paid).
- Colab (convenient; timings can be noisy).

Colab GPU how-to: enable GPU guide.

10.3 What you report

- Confirm CUDA version matches serial accuracy on a fixed subset.
- Report GPU vs CPU time for the same workload and batch sizes.
- State GPU model and environment in the report.

11. Experiment matrix (what to run for plots and tables)

11.1 Accuracy table (required)

Create one table: serial vs OpenMP vs pthreads vs MPI vs hybrid vs CUDA. Show Q3 with the same split and hyperparameters.

11.2 Timing sweeps (required)

Run timing experiments with fixed epochs and fixed batch size so comparisons are fair.

Variant	Knobs to sweep	Suggested sweep	What to plot
Serial	None	1 run	Baseline time
OpenMP	Threads	1, 2, 4, 8, 16	Time vs threads; Speedup vs threads
Pthreads	Threads	1, 2, 4, 8, 16	Time vs threads; Speedup vs threads
MPI	Ranks (laptops)	1, 2, 3	Time vs ranks; Speedup vs ranks
Hybrid	Ranks x threads	(1x1,1x8,1x16, 2x8,2x16, 3x8,3x16)	Heatmap/table; best config
CUDA	Batch size	32, 64, 128, 256	Time vs batch; GPU vs CPU

12. Report template (what to write, in order)

12.1 Recommended report outline

- Introduction: problem definition and motivation.
- Dataset + preprocessing: CB513 provenance, label mapping, windowing, encoding.
- Serial baseline: model architecture, training setup, Q3 definition.
- Parallel designs (with diagrams): OpenMP, pthreads master/worker, MPI, hybrid, CUDA.
- Results: accuracy table; timing plots; speedup/efficiency plots; bottleneck discussion.

- Conclusion: best-performing approach and lessons learned.

12.2 Diagrams you must include (high marks)

- OpenMP: data-parallel gradient computation + reduction.
- Pthreads: master/worker flow (Section 7 diagram).
- MPI/Hybrid: ranks + Allreduce + intra-rank threading (Section 9 diagram).

13. Common pitfalls and how to avoid them

- Timing includes preprocessing: preprocess once; benchmark on preprocessed binary data.
- Parallel accuracy differs: fix RNG seeds and compare Q3 every run.
- OpenMP overhead: avoid tiny repeated parallel regions; keep a persistent region.
- False sharing in pthreads: keep per-thread buffers separate and cache-aligned.
- Oversubscription in hybrid: keep ranks×threads <= 16 per laptop.
- MPI timing: barrier before timing; report the slowest rank time per epoch.

14. Links and resources

These are the key external resources to cite and/or use during implementation:

Paper (precedent)	Zhong et al. (2007) DOI 10.1007/s11227-007-0100-1	https://link.springer.com/article/10.1007
CB513 original page	JPred legacy datasets (CB513)	https://www.compbio.dundee.ac.uk/jpred/legacy/data/pred_res
CB513+profiles	Ready-to-use NumPy arrays	https://github.com/taneishi/CB513_dataset
DSSP	Secondary-structure assignment reference	https://pdb-redo.eu/dssp
OpenMP	Specifications	https://www.openmp.org/specifications/
Pthreads	man7 pthreads(7)	https://www.man7.org/linux/man-pages/man7/pthreads.7.html
Pthreads tutorial	LLNL HPC tutorial	https://hpc-tutorials.llnl.gov/posix/
OpenMPI	mpirun manual	https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man1/mpiru
CUDA	CUDA Toolkit docs	https://docs.nvidia.com/cuda/
cuBLAS	BLAS on CUDA	https://docs.nvidia.com/cuda/cublas/index.html
Colab GPU	Enable GPU guide (practical)	https://www.geeksforgeeks.org/python/how-to-use-gpu-in-goog